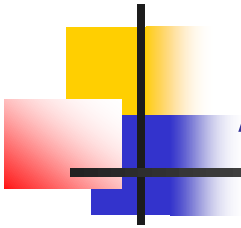
Projeto e Análise de Algoritmos

Algoritmos Gulosos (Parte I)

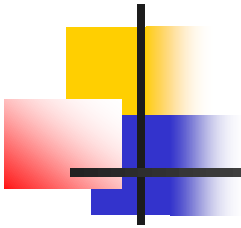
Universidade Federal do Amazonas
Instituto de Computação





Algoritmos Gulosos

- Quando aplicados a otimização, utilizam o mesmo conceito de subestrutura ótima usado na prog. Dinâmica.
- Fazem escolhas com base em dados locais para encontrar boas soluções
- Técnica pode ser usada para obter resultados aproximados em alguns casos

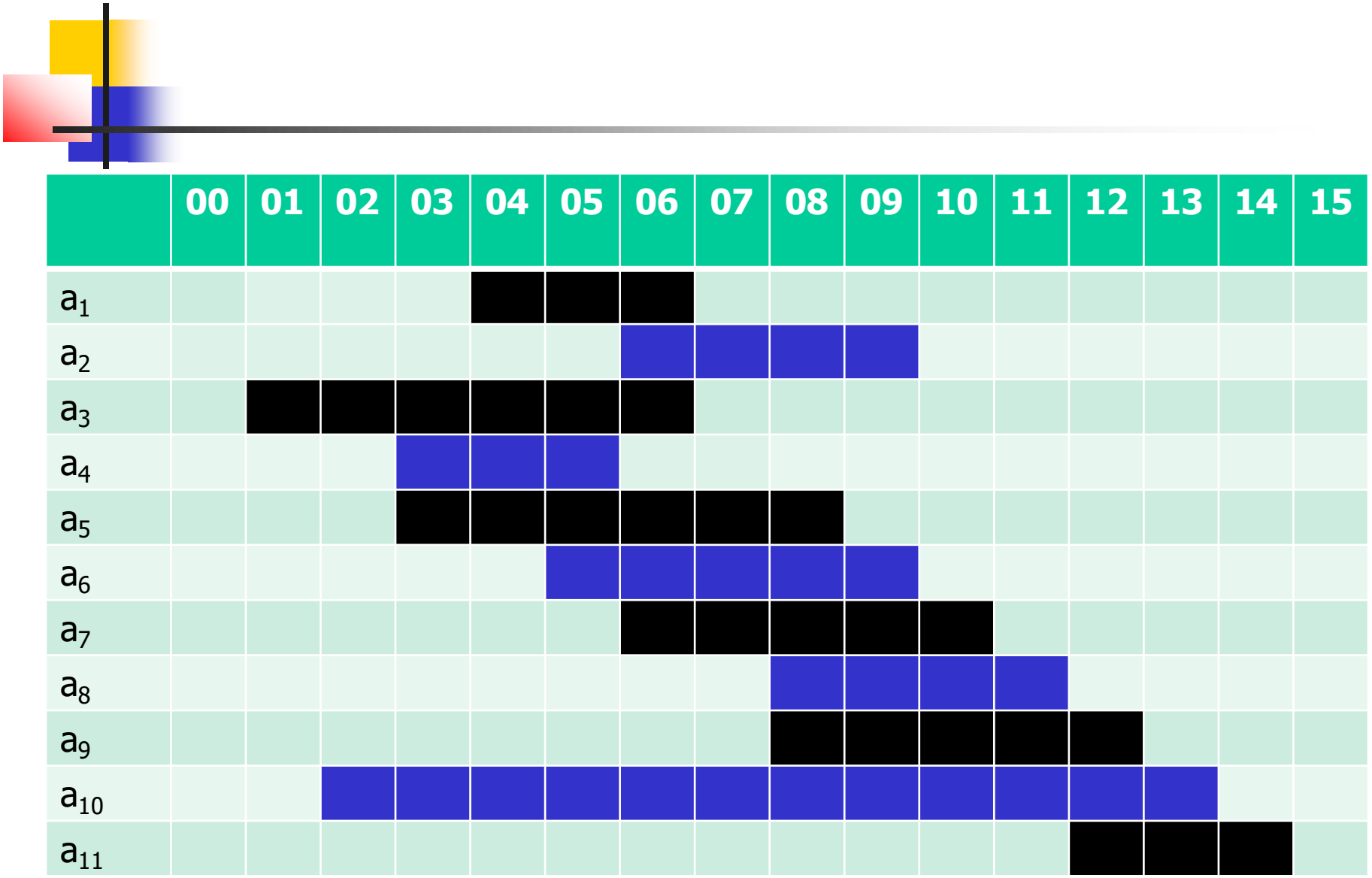


Seleção de Atividades

- Tenho um recurso e diversas atividades concorrentes que se pretende executar utilizando este recurso. Cada atividade i começa no tempo si e termina no tempo fi
- Devemos selecionar o máximo possível de atividades compatíveis

Exemplo

i	1	2	3	4	5	6	7	8	9	10	11
si	4	6	1	3	3	5	6	8	8	2	12
fi	6	9	6	5	8	9	10	11	12	13	14


00
01
02
03
04
05
06
07
08
09
10
11
12
13
14
15
a1
a2
a3
a4
a5
a6
a7
a8
a9
a10
a11

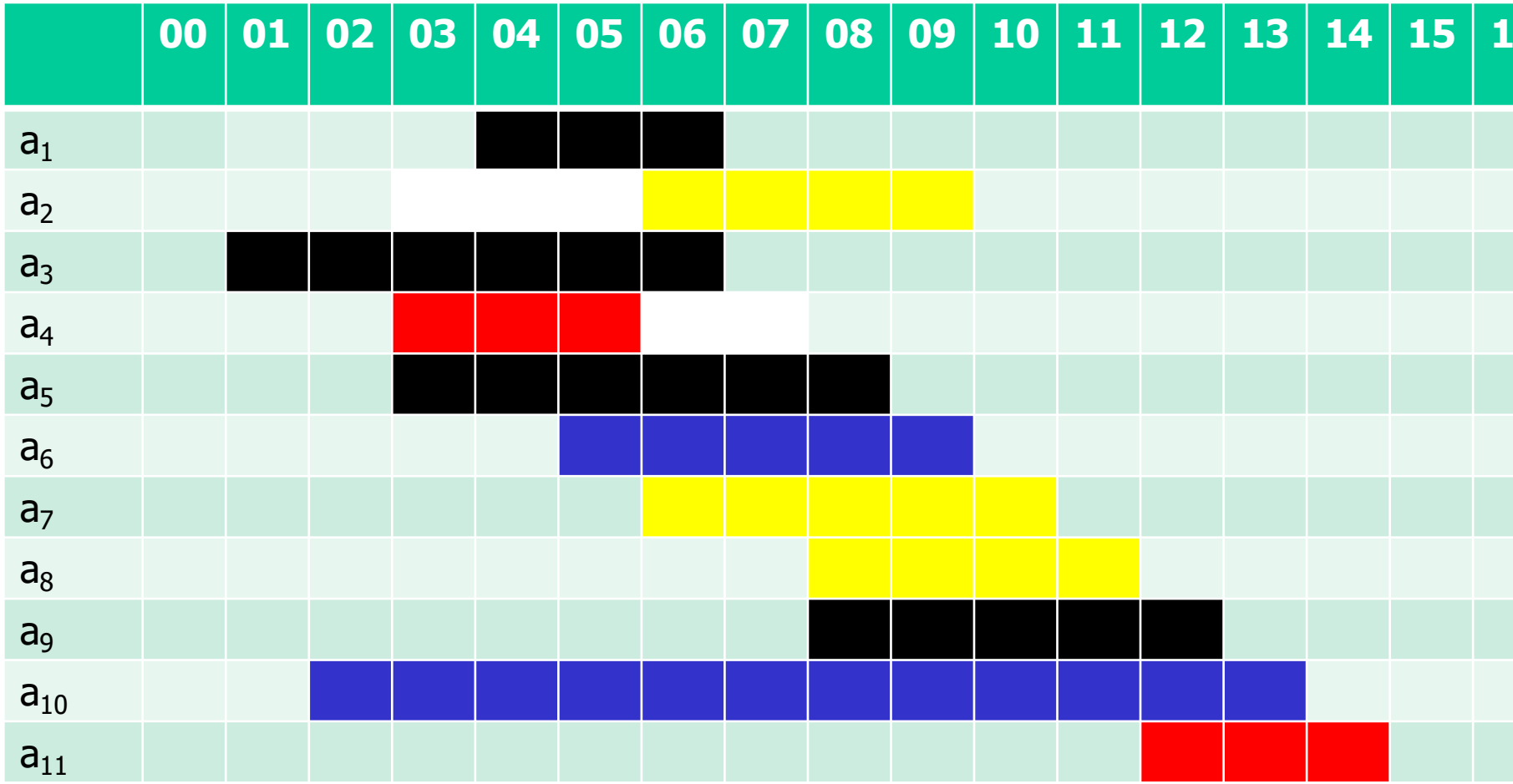
Espaço de subproblemas

$$S_{i,j} = \{a_k \in S : f_i \leq s_k < f_k \leq s_j\}$$

Conjunto das atividades que podem começar após o fim de a_i e terminar antes de a_j começar

Atividades que estão “entre” ai e aj.
Se considerarmos atividades ordenadas por tempo de término Si,j vazio para i>=j (não haveria atividades entre ai e aj nestes casos)

$S_{4,11}=\{a_2, a_7, a_8\}$


00
01
02
03
04
05
06
07
08
09
10
11
12
13
14
15
a1
a2
a3
a4
a5
a6
a7
a8
a9
a10
a11

Subestrutura ótima

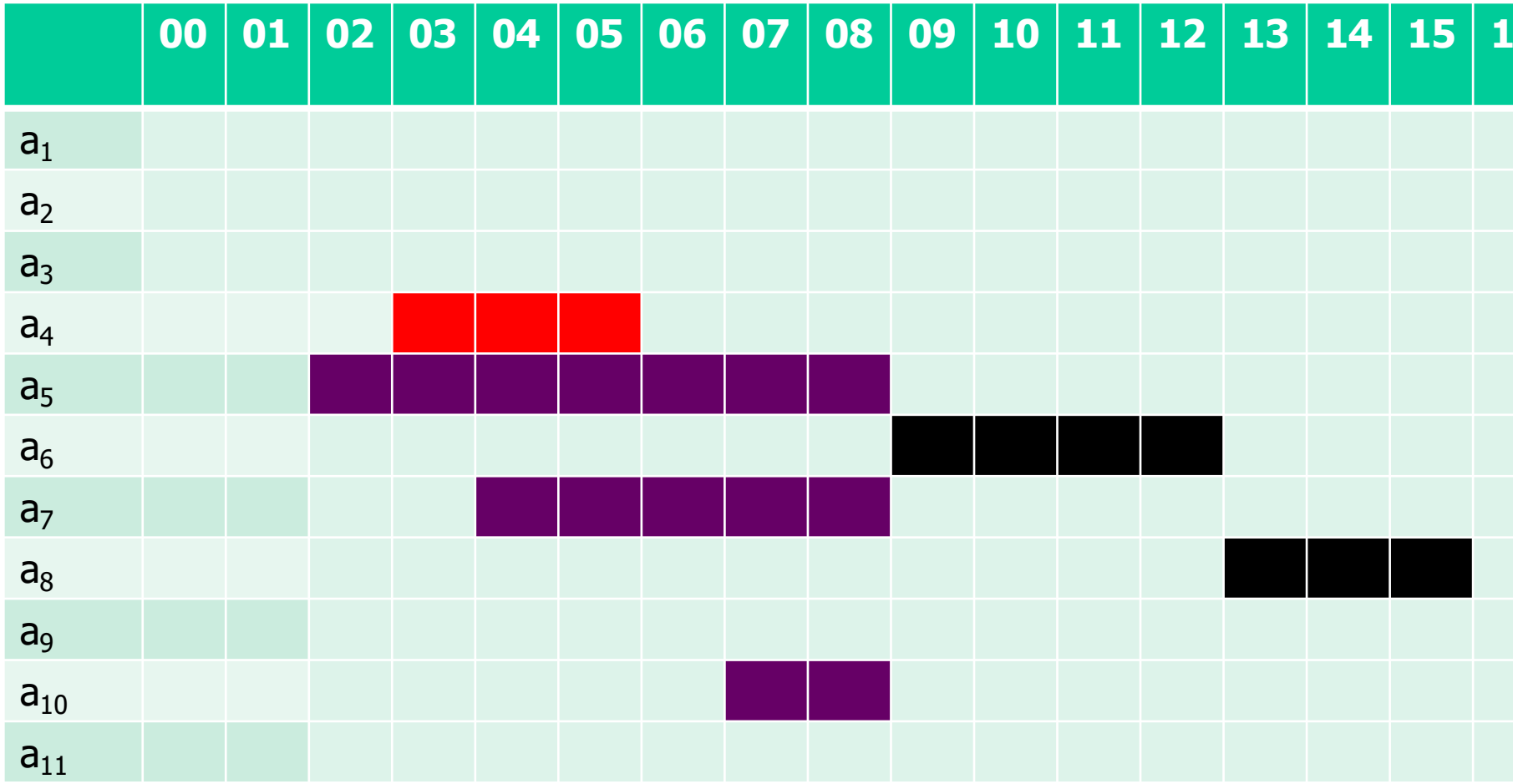
$$c[i,j] = \begin{cases} 0 & \text{se } s_{i,j} \text{ vazio} \\ \max_{i<k<j}\{c[i,k]+c[k,j]+1 \end{cases}$$

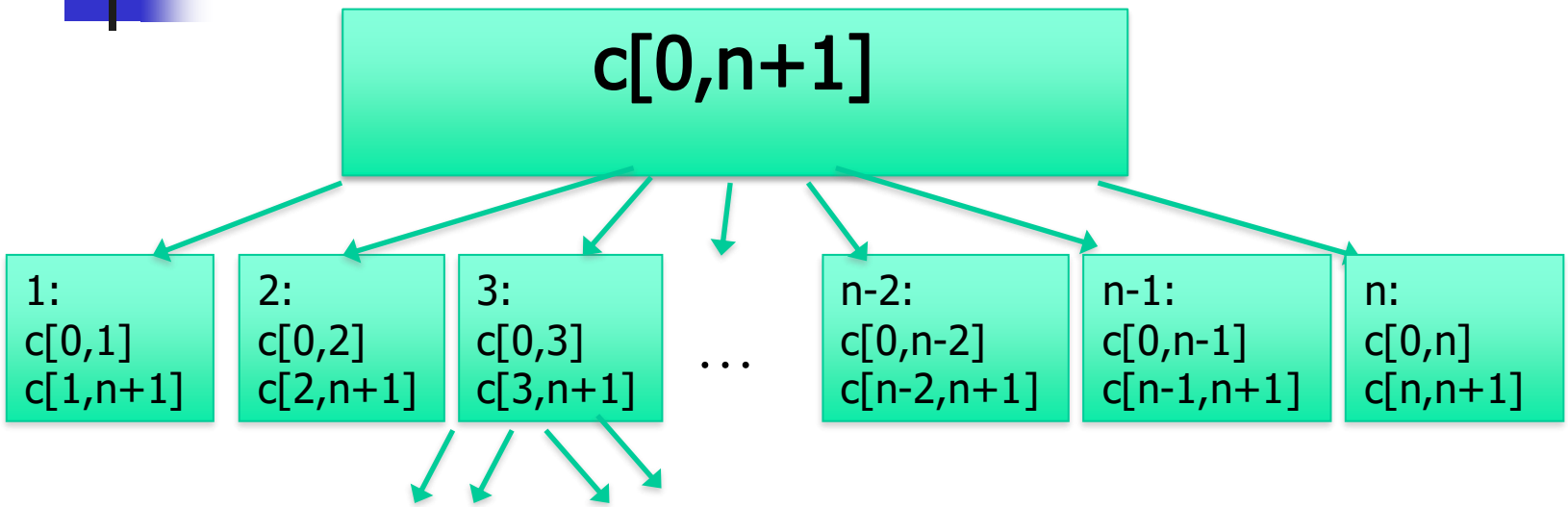
Problema formulado recursivamente c[i,j] é o maior número de atividades (entre) ai e aj.

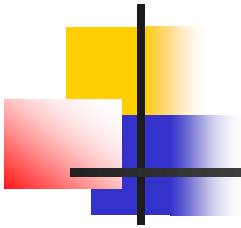
Começamos com c[0,n+1] para obter solução

a_0 termina no tempo 0 e a_{n+1} começa no infinito

$S_{4,11}=\{a_2, a_7, a_8\}$


00
01
02
03
04
05
06
07
08
09
10
11
12
13
14
15
a1
a2
a3
a4
a5
a6
a7
a8
a9
a10
a11

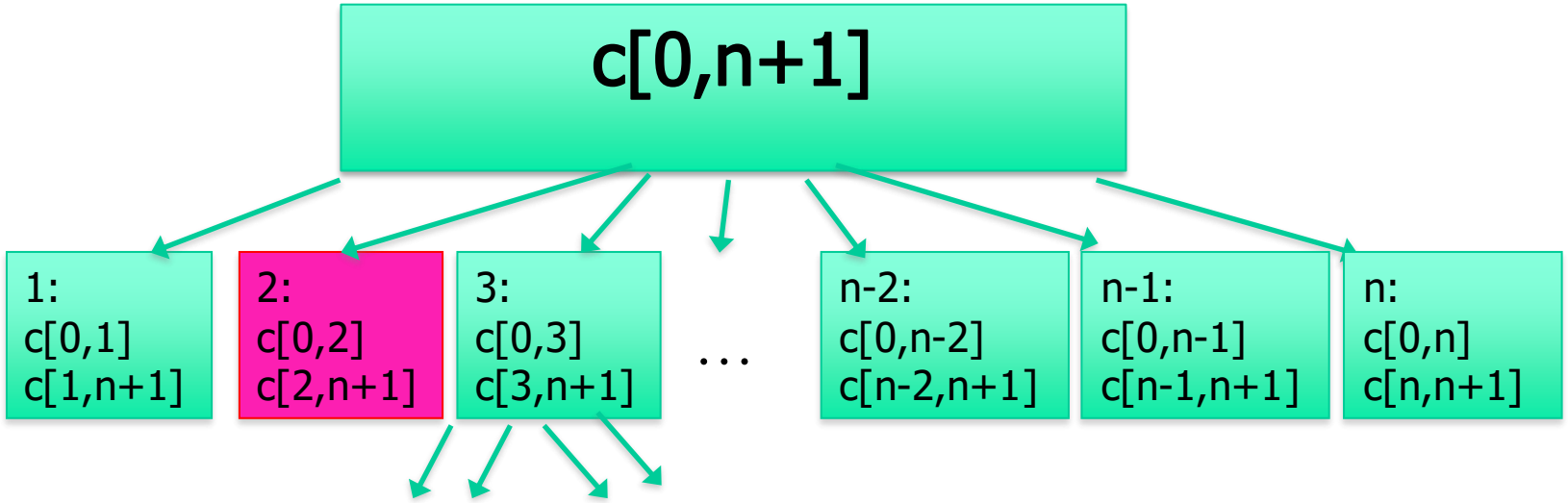

c[0,n+1]
1:
c[0,1]
c[1,n+1]
2:
c[0,2]
c[2,n+1]
3:
c[0,3]
c[3,n+1]
...
n-2:
c[0,n-2]
c[n-2,n+1]
n-1:
c[0,n-1]
c[n-1,n+1]
n:
c[0,n]
c[n,n+1]

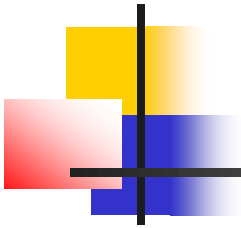


Solução gulosa

- A atividade que termina primeiro faz parte de algum subconjunto de número máximo de atividades mutuamente compatíveis com $S_{i,j}$
- Sendo m a atividade que termina primeiro
- O subproblema $S_{i,m}$ é vazio (confira!)

Guloso versus força bruta

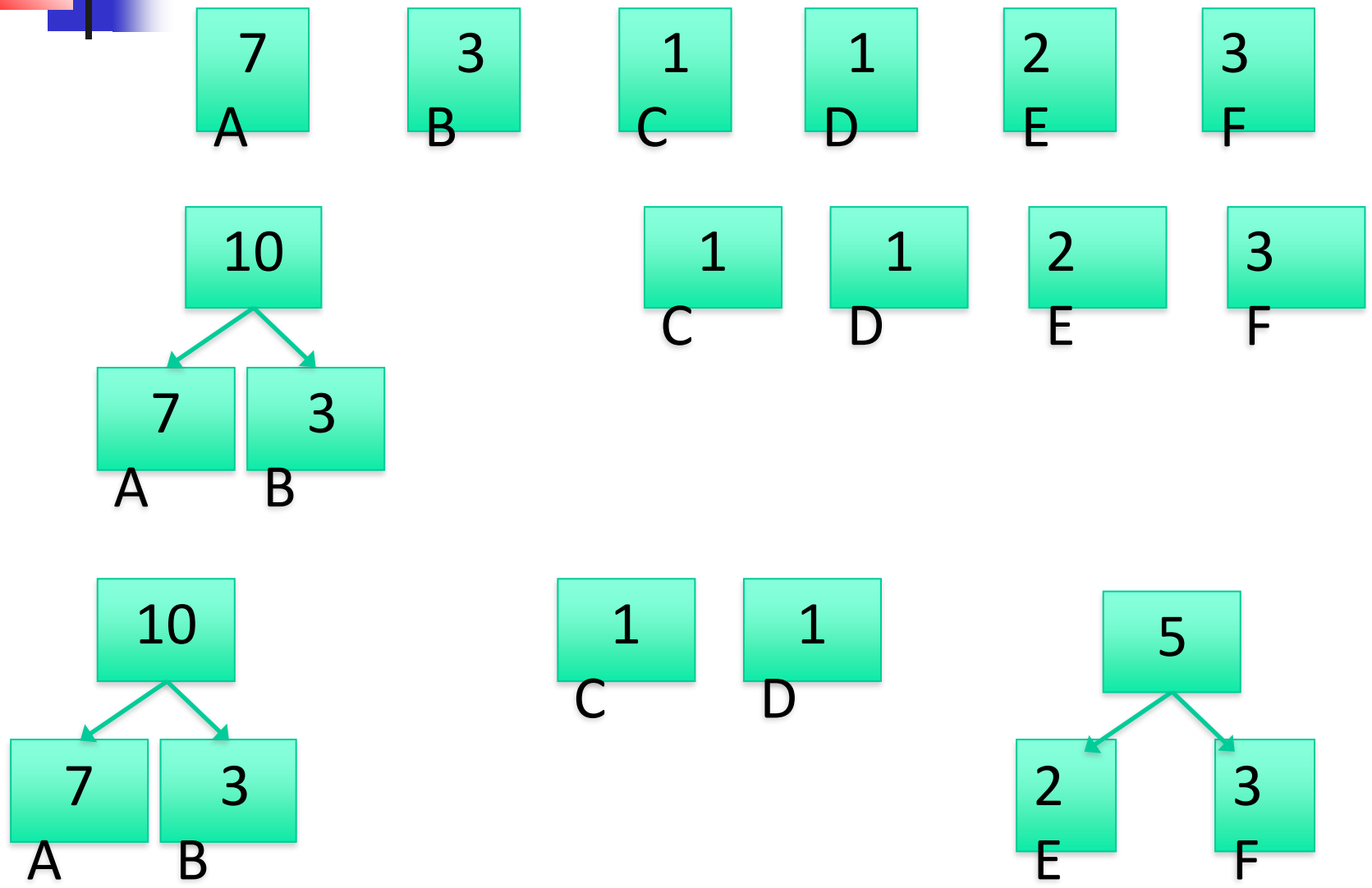




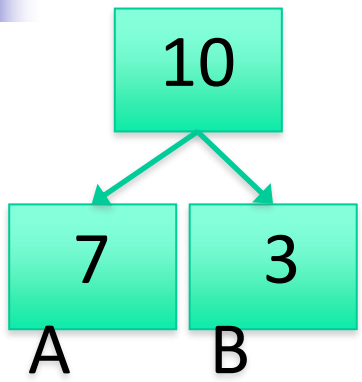
Exemplo: Códigos de Prefixo

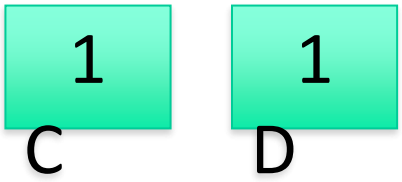
- Codificação dada a um conjunto de símbolos onde nenhum símbolo é prefixo de outro
- Exemplos: Considere os símbolos s1,s2, s3 e s4
- {001,00,01,1} não é código de prefixos
- {001,010,011,1} é um código de prefixos

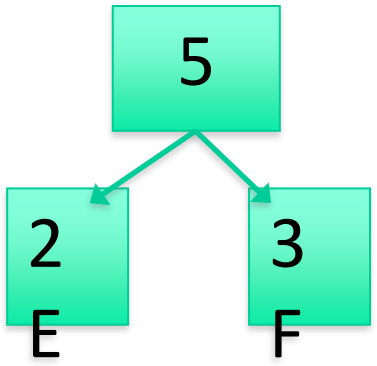
Árvore de codificação

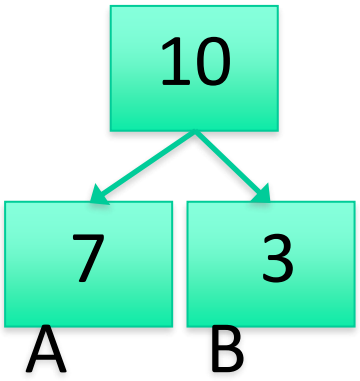


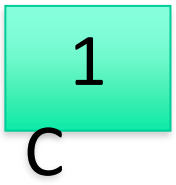
Arvore de codificação de prefixos

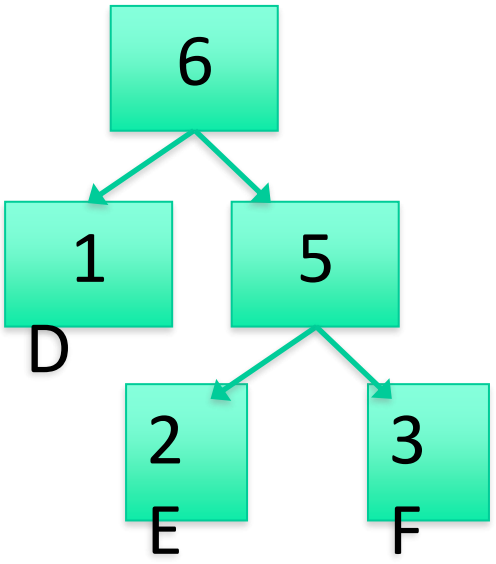


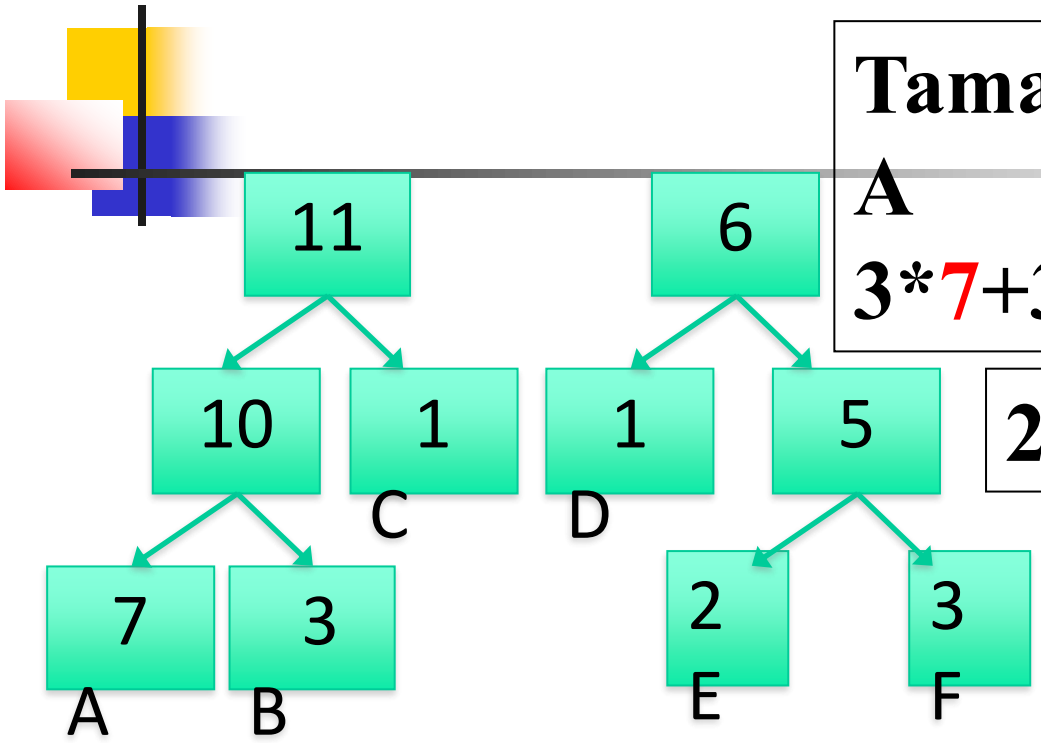












Tamanho texto codificado

A	B	C	D	E	F
3*7	+3*3	+2*1	+2*1	+3*2	+3*3

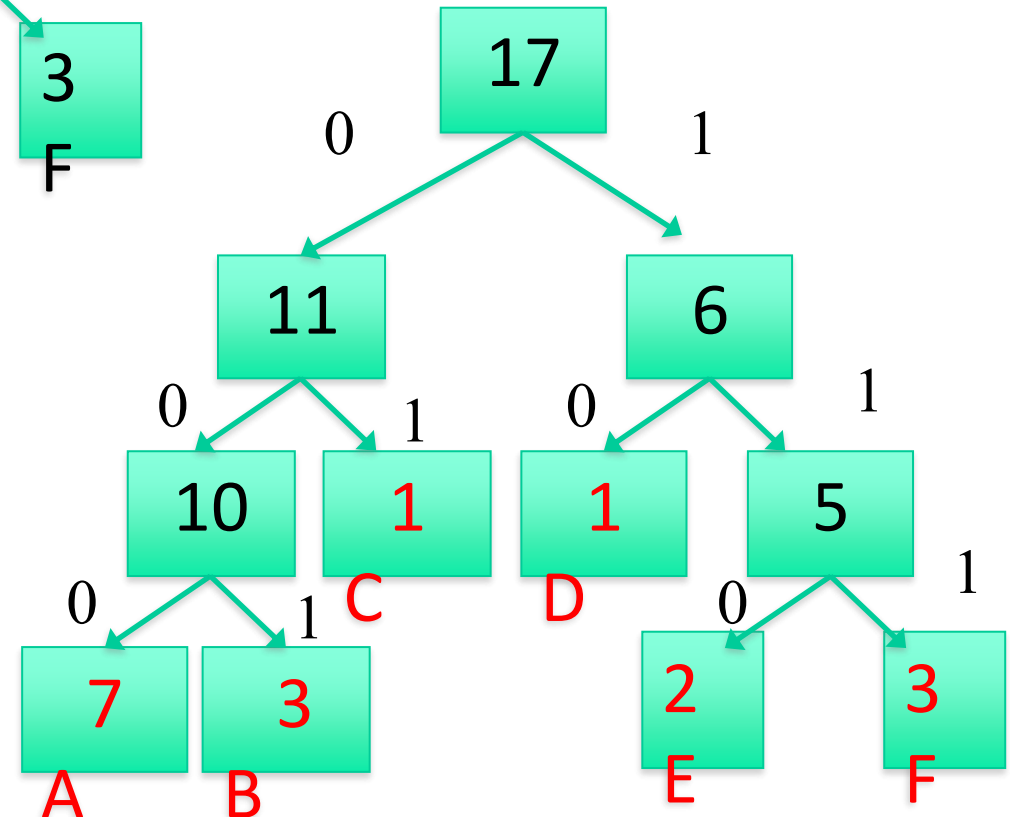
21+9+2+2+6+9=49 bits

Códigos:

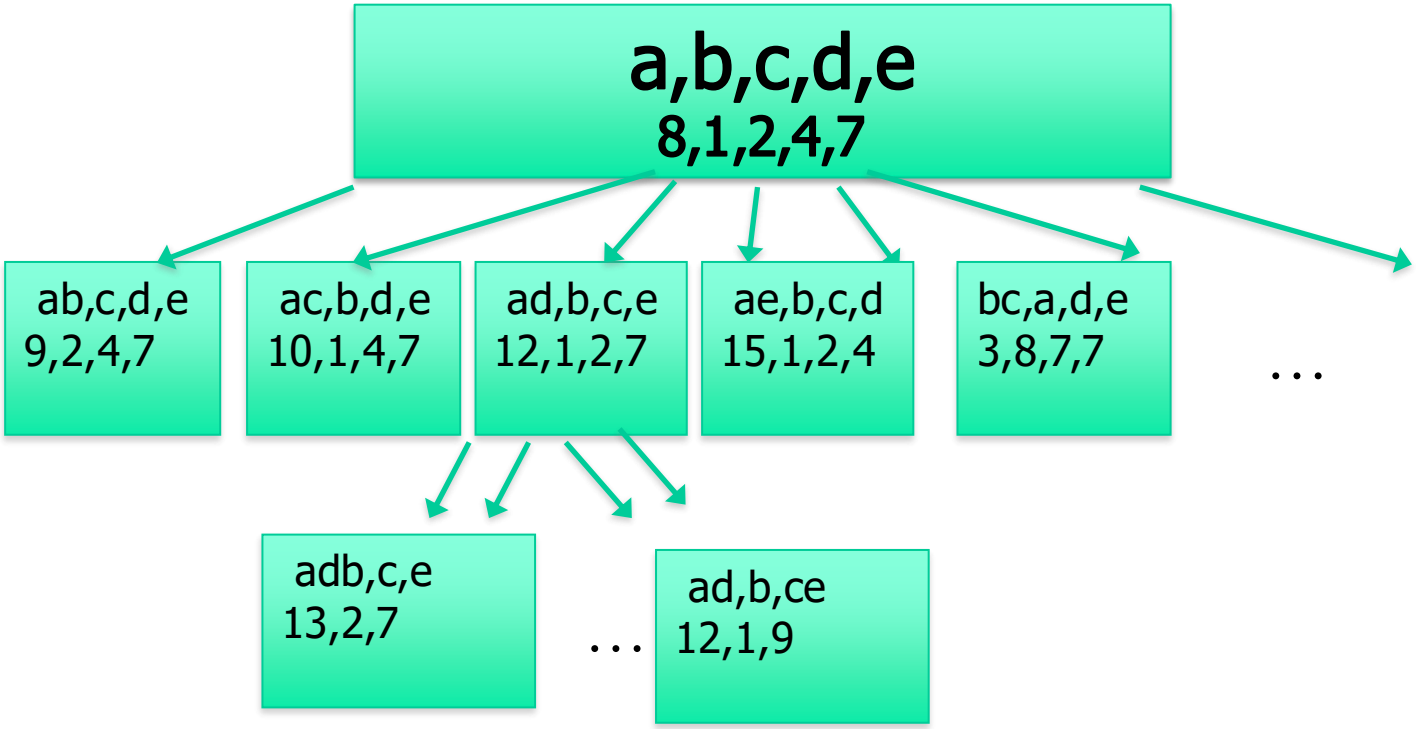
A-000 B-001

C-01 D-10

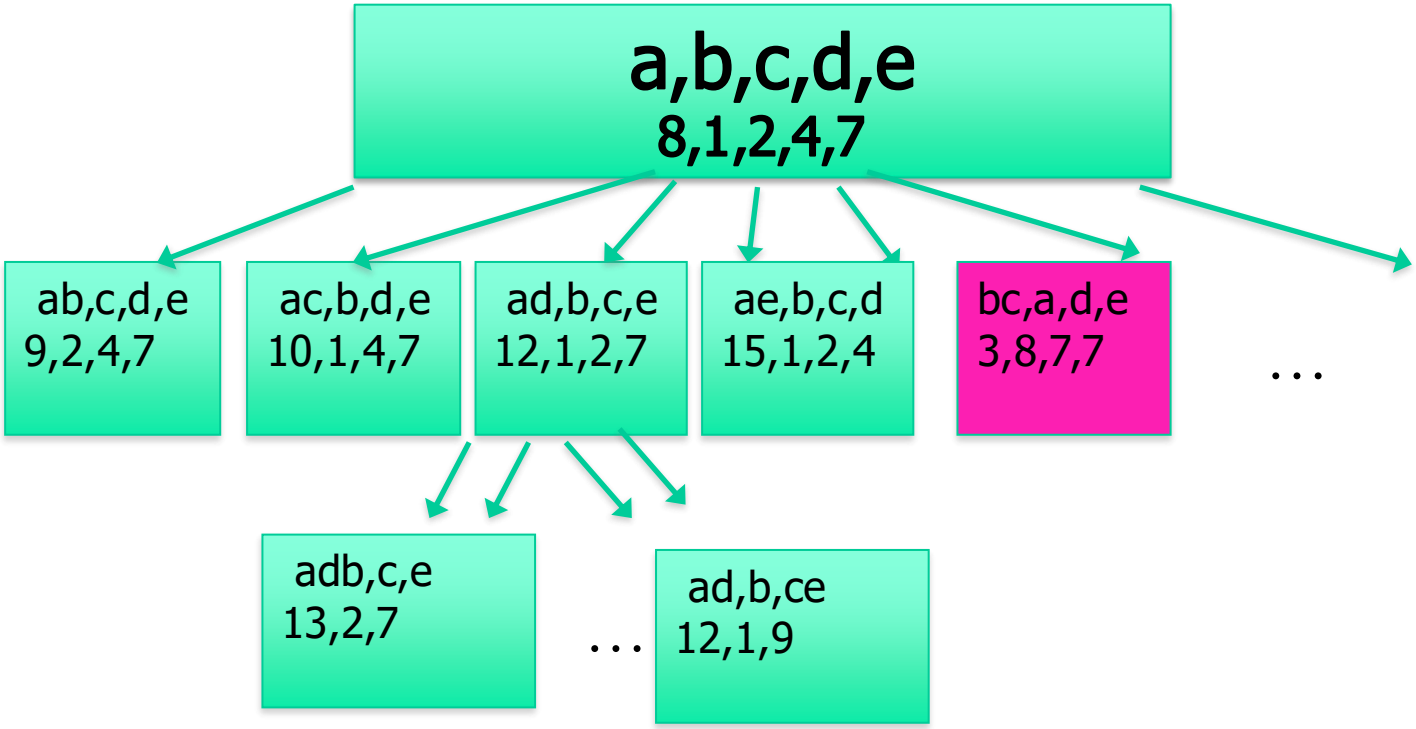
E-110 F-111



Força bruta


a,b,c,d,e
8,1,2,4,7
ab,c,d,e
9,2,4,7
ac,b,d,e
10,1,4,7
ad,b,c,e
12,1,2,7
ae,b,c,d
15,1,2,4
bc,a,d,e
3,8,7,7
...
adb,c,e
13,2,7
...
ad,b,ce
12,1,9

Alguma escolha gulosa...



Código de prefixos de redundância mínima

- Dada uma fonte de dados com símbolos S={s1,s2,..,s_v} e suas respectivas freqüências F={f1,f2,..,f_v}, determine o código de prefixos que minimize a função

$$C(S,F) = \sum_{i=1}^{v} | s_i | f_i$$



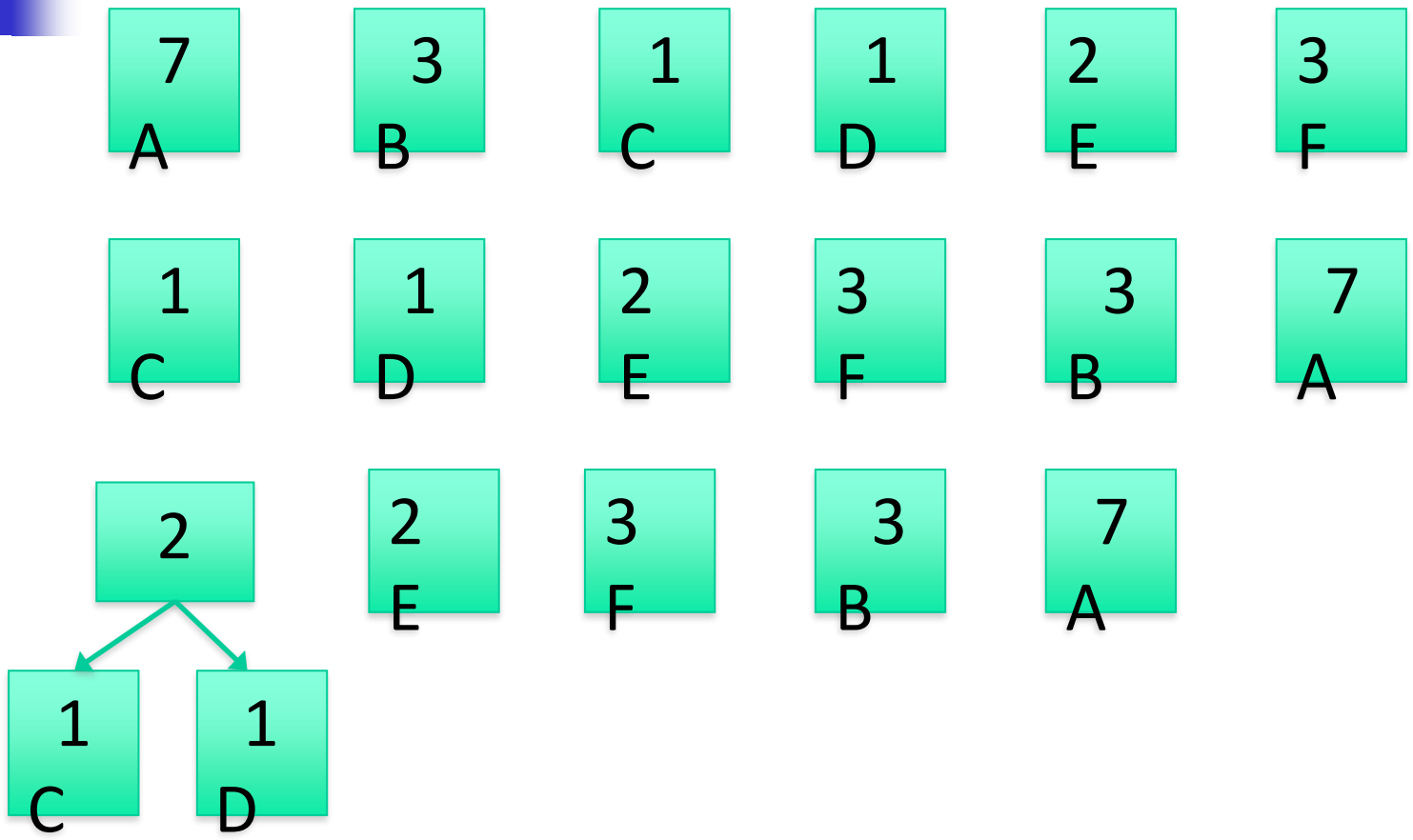
Algoritmo de Huffman

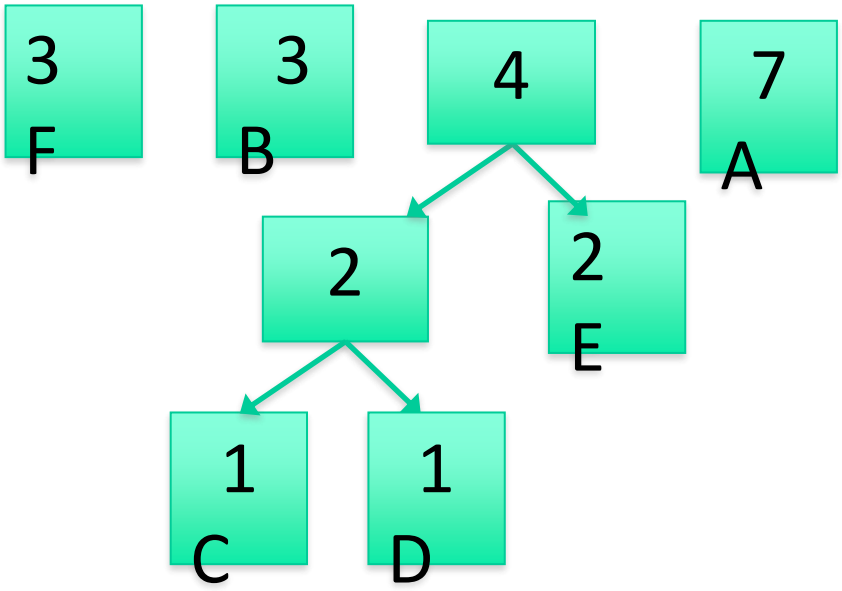
- Dado um conjunto de símbolos com suas respectivas probabilidades de ocorrências constrói-se uma árvore de codificação das folhas para a raiz.
- inicialmente cada símbolo é representado por um nó que é tratado como uma árvore individual
- cada nó recebe um peso equivalente a probabilidade de ocorrência do símbolo correspondente

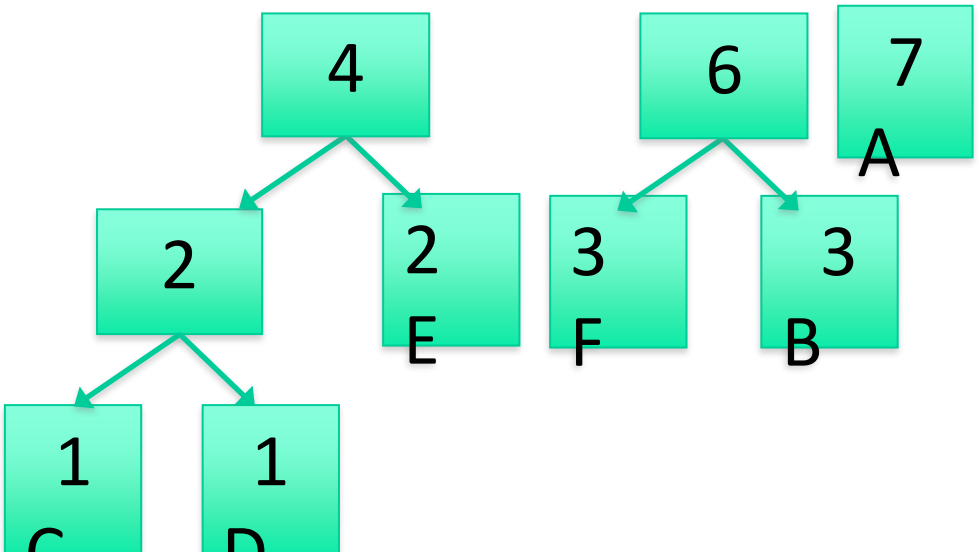
Algoritmo de Huffman

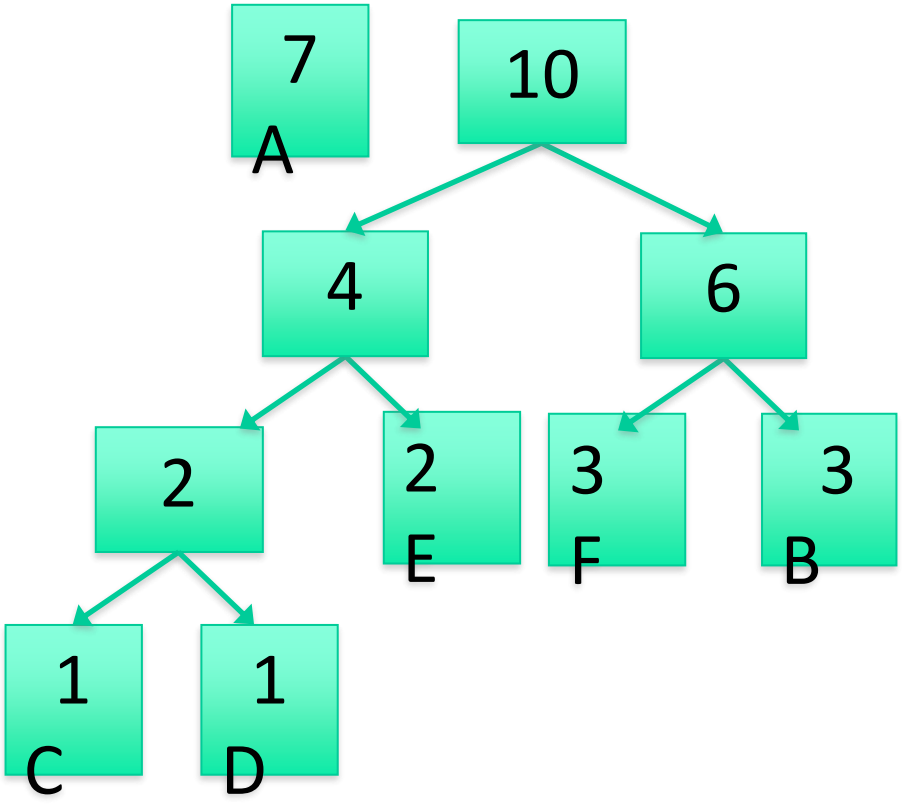
- **ESCOLHA GULOSA:** As duas árvores de menor peso são então unidas por um nó interno que recebe peso igual a soma das probabilidades das árvores unidas
- O processo é repetido até que reste somente uma árvore, a qual é conhecida como árvore de Huffman

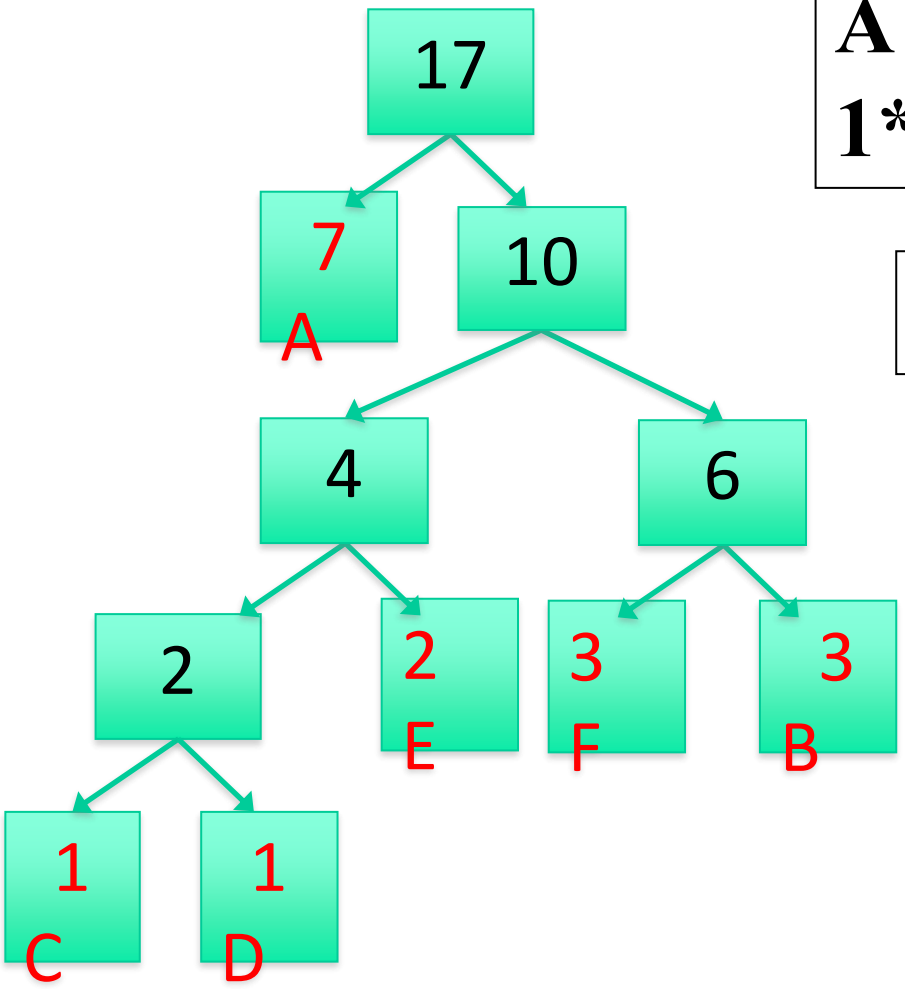
Huffman




3
F
3
B
4
7
A
2
2
E
1
C
1
D


4
6
7
A
2
2
E
3
F
3
B
1
1


7
A
10
4
6
2
2
E
3
F
3
B
1
C
1
D



Tamanho texto codificado

A B C D E F

1*7+3*3+4*1+4*1+3*2+3*3

7+9+4+4+6+9=39 bits

Códigos:

A-0 B-111

C-1000 D-1001

E-101 F-110



- Código de Huffman é ótimo.
- Pode-se provar assumindo-se que não é e mostrando que leva a um absurdo.